

Fast first-factor finding function

Document #: P3133R0
Date: 2024-02-14
Project: Programming Language C++
Audience: Library Evolution Working Group
SG6 Numerics
Reply-to: Chip Hogg ([Aurora Innovation](#))
<charles.r.hogg@gmail.com>

Contents

- [1 Introduction](#)
- [2 Use case: magnitudes for units](#)
 - [2.1 In more detail: the vector space representation](#)
 - [2.1.1 Current workaround: “known first factor”](#)
- [3 Proposal](#)
 - [3.1 Compiler iteration limits](#)
 - [3.2 Usage examples](#)
 - [3.3 Open questions](#)
 - [3.3.1 Contract for numbers less than 2](#)
 - [3.3.2 Contract for negative numbers](#)
 - [3.3.3 Larger integer types](#)
- [4 What if we don’t add this?](#)
- [5 Conclusion](#)
- [6 References](#)

§ 1 Introduction

We propose a new standard library function, `std::first_factor`, that returns the smallest prime factor of a given number. The goal is to efficiently produce robustly correct results for any input of any standard integer type. We also require it to be `constexpr` compatible, because one core motivating use case needs the results at compile time.

Integer factorization is among the most thoroughly studied problems in computer science; the wikipedia page lists roughly a dozen commonly used algorithms. The ideal algorithm would have three properties: it would be **efficient** (in both time and space complexity), **robust** (that is, reliably producing the correct answer for any input), and **easy to implement**.

Unfortunately, each of these algorithms offers at most two of these properties. For example, trial division is both robust and easy to implement, but is unacceptably slow for inputs whose first factor is large. Pollard's rho algorithm is efficient and easy to implement, but is not robust: for any given choice of parameters, there are inputs that fail to produce an answer.

The solution is to take implementation off the table. If the standard library provides a robust and efficient implementation, the end user experience will be just as good as if we had an algorithm with all three properties.

An existence proof is readily available. The unix program `factor` can fully factor any 64-bit number in a tiny fraction of a second. The program source code is also written in C, so it would be easy to produce a C++ implementation.

The immediate motivation is that this would unblock the currently active proposal for a standard units library, described in [\[P3045\]](#). However, factoring integers is such a common and basic operation — and the proposed function such a small addition — that we believe its inclusion would be well justified by the wide variety of use cases it would support, many of which we cannot easily anticipate.

§ 2 Use case: magnitudes for units

The connection between integer factorization and a units library may not be immediately obvious. We'll begin with a brief outline, which should suffice to motivate the feature. We'll then add further details for the curious reader.

What we need is a software representation for the *magnitude* of a unit — that is, the property that is 12 times as large for a foot as for an inch. This representation must support all of the same operations that units support. It turns out that these are **products** and **rational powers** (which, when combined, also give us quotients, inverses, and roots). We can satisfy this requirement by representing each integer magnitude as a product of powers of primes. Here is how to perform those operations in this representation:

- For products, we simply add the exponents for each prime factor.
- For rational powers, we simply multiply each exponent by the power.

This representation has been shown to work very well in practice, across multiple widely used C++ units libraries ([\[mp-units\]](#) and [\[Au\]](#)). The bottleneck is factoring numbers that contain a very large prime factor. Consider a robust but inefficient factorization method, such as trial division, or its wheel factorization enhancement. While this does take a very

long time, there's an even bigger problem. Compilers *limit the number of iterations* in a `constexpr` loop, and these algorithms can easily exceed this limit for large inputs: we get a *compilation error* rather than an excessively slow compilation.

This problem is more than just theoretical: for example, the exact definition of the proton mass in SI units involves the number 1,672,621,923,695, one of whose factors is 334,524,384,739, which is prime. We discovered this problem when we realized that the proton mass could not be defined in the `mp-units` library, because it exceeded the compiler's iteration limits on every major compiler.

We could solve this problem for our own units libraries by deeply studying state of the art factorization approaches, ones that provide correct answers quickly for any input. We would of course also need to develop a rigorous battery of tests, to give us confidence that we've implemented these tricky and subtle algorithms correctly. And we could ask every other project with similar needs to do the same.

Or, we could add a rigorous, well-tested implementation to the C++ standard library, behind a simple and easily understood interface, letting us and every other project redeploy our attention to more interesting problems.

§ 2.1 In more detail: the vector space representation

Here, we'll explore the units library need for integer factorization in more detail, with the goal of understanding why simpler approaches are inadequate. Readers who are happy to accept the need for the product-of-powers-of-primes representation for argument's sake can skip this section on a first reading.

The most familiar simpler alternative is of course `std::ratio`. This utility was introduced to support the `std::chrono` library, which is a kind of single-dimensional (time-only) units library. Unfortunately, moving to a multi-dimensional units library brings new, more stringent requirements, and `std::ratio` fails to meet them in multiple ways.

The main new requirement for a multi-dimensional units library is additional pathways for making new units. In a single-dimensional library, we can only multiply and divide units by scalar quantities (for example, dividing the “hour” by 60 to obtain the “minute”). Multi-dimensional libraries let you make new units by multiplying or dividing existing ones (for example, the “meter” divided by the “second” produces the “meter per second”, a unit of speed). You can also raise units to powers, even rational ones (for example, the “liter” raised to the power $1/3$ is the “decimeter”).

What these new pathways have in common is that they *change the dimension* of the input units. This explains why these operations don't arise in single-dimensional units libraries such as `std::chrono`: they would exit the library's domain.

In the previous section, we mentioned that the new representation — the product of powers of primes — makes products and rational powers easy to compute. Of course, we could obtain this same benefit without performing prime factorization. However, there's

another requirement to consider: each number must have a *unique* representation. Otherwise, the same unit could be represented by many different types. This is the added value of choosing prime numbers: they also provide uniqueness, because the fundamental theorem of arithmetic guarantees that every integer has a unique factorization into prime numbers.

Another benefit comes from the fact that every number will be decomposed onto the *same* set of elements. Abstracting a little bit, we can think of the prime numbers as a set of common “basis” elements. Interestingly, it turns out that this representation satisfies all of the axioms of a vector space:

- The **prime numbers** play the role of **basis vectors** (and are indeed but one choice among many!)
- The **rational powers** play the role of **scalar multiplication**
- The **product** operation plays the role of **vector addition**

For this reason, we refer to this representation as the “vector space” representation. See the [\[vector space discussion\]](#) for more details on this connection.

While this strategy may come across as very abstract and theoretical, it has direct practical implications. When we upgraded the mp-units library from a `std::ratio`-like representation to the vector space alternative, it solved *three seemingly unrelated pre-existing issues*.

1. We could not represent rational powers of units. (For example, this can arise when computing the average particle speed, in meters per second, in a Maxwell-Boltzmann distribution, where the particle mass is represented in Daltons.)
2. Even small powers of certain units can be excessively vulnerable to overflow. (For example, the area unit “Astronomical Unit Squared” cannot be represented.)
3. We could not simultaneously represent units whose quotient is transcendental. (For example, degrees and radians differ by a factor of π , and a `std::ratio`-like approach could not handle π .)

The reason the vector space representation solves this last problem is that we can include π as a new basis vector, alongside the primes. Recall that a basis must be linearly independent. For this kind of vector space, the “linear combination” operation maps onto a product of (rational) powers of basis elements. Thus, for π to be a new basis element, we require that no product of powers of primes can produce π . Happily, this follows directly from its transcendental nature.

The vector space representation is the only known representation that satisfies all of the requirements of a multi-dimensional units library. It critically depends on the ability to fully factor integers at compile time. Moreover, even simple practical use cases (such as the proton mass) can exceed the capabilities of the factorization algorithms that are easy to implement with high confidence. This is the motivation for adding a simple, easily-

understood API to the standard library, which can encapsulate a robust, well-tested implementation.

§ 2.1.1 Current workaround: “known first factor”

As it happens, the `mp-units` *does* provide the proton mass. However, the mechanism is a hack that is completely unsuitable for standardization.

We provide a template trait, `known_first_factor<N>`, which the library (or even end users!) can specialize. The factorization algorithm checks this trait first, and if it finds a specialization, it simply trusts that it is correct, and short-circuits the computation. For example, here’s the specialization that enables us to define the proton mass:

```
template<>
inline constexpr std::optional<std::intmax_t>
mp_units::known_first_factor<334'524'384'739> = 334'524'384'739;
```

In this case, the first factor is the number itself, because it happens to be prime. The reason we chose `known_first_factor` instead of something like `is_prime` is because the former would also allow us to handle a product of multiple large primes, each of which would exceed compile-time iteration limits on its own.

The downsides of this approach are clear. If the specialization provides the wrong answer, the result is undefined behavior. When end users specialize this trait, they also need to figure out where to put this specialization so that it will be visible everywhere it’s needed. It would be far better and simpler to provide an implementation that can compute correct values for every input, especially given that the `unix program factor` already shows that this is possible.

§ 3 Proposal

The core requirement for the `std::first_factor` API is that the following signature exist:

```
constexpr std::uint64_t std::first_factor(std::uint64_t n);
```

Whenever $n > 1$, this must return the smallest prime factor of n , and do so as efficiently as possible.

There may be other overloads besides the above for `std::uint64_t`, as we’ll discuss below. However, in every case, the return type should be the same as the input type, because the result of the function can clearly never exceed its input.

§ 3.1 Compiler iteration limits

The compiler iteration limits that we mentioned above present an additional complication. We have already seen that there are inputs that exceed these limits for naive algorithms, preventing compilation. We don’t yet know whether there are any inputs that would

exceed the limits for the more efficient algorithms that we would like to use. (After all, avoiding the need to research, tune, and implement these algorithms ourselves was a major reason for writing this paper!)

The pessimistic outcome would be that every possible “pure C++” implementation has at least one input that would exceed these iteration limits for some compiler. If that happens, the implementation will probably need help from compiler internals. We don’t expect this to be a problem, because there’s precedent for this with some type traits (see, for example, [\[is_trivially_constructible discussion\]](#)).

§ 3.2 Usage examples

Here is one example of how to factorize an unsigned integer using this function:

```
std::vector<std::uint64_t> factorize(std::uint64_t n) {
    std::vector<std::uint64_t> factors;
    while (n > 1) {
        factors.push_back(std::first_factor(n));
        n /= factors.back();
    }
    return factors;
}
```

This next example is much more complicated, but also more practically useful: we’ll factorize integers again, but this time at compile time. We’ll store the result in a variadic template, whose parameters are rational powers of primes. That is, the goal is for, say, `PrimeFactorization<1176u>` to be an alias that resolves to `Magnitude<BasePower<2, std::ratio<3>>, BasePower<3>, BasePower<7, std::ratio<2>>>`. Here’s one possible implementation using `std::first_factor`.

```
// `Magnitude<BasePower<N, Exp>, ...>` is a representation of a positive
real
// number, as a product of powers of "basis elements". In this case, the
basis
// elements are the prime numbers, although a more sophisticated
implementation
// would add other basis elements such as pi.
template <typename... BasePowers>
struct Magnitude {};

// `BasePower<N, Exp>` represents one basis element raised to a rational
power.
template <std::uint64_t Base, typename Exp = std::ratio<1>>
struct BasePower {};

// `Concatenate<C1, C2>` is a metafunction that concatenates two type
lists, as
// long as they are instances of the same variadic template template.
template <typename C1, typename C2>
struct Concatenate;
template <template <typename...> typename C, typename... Ts, typename...
    Us>
struct Concatenate<C<Ts...>, C<Us...>> {
```

```

    using type = C<Ts..., Us...>;
};

// Divide `n` by `factor` as many times as possible, and return two
values:
// the number of divisions, and the amount remaining afterwards.
constexpr std::pair<std::uint64_t, std::uint64_t> extract_power(
    std::uint64_t n, std::uint64_t factor) {
    std::uint64_t power = 0;
    while (n % factor == 0) {
        ++power;
        n /= factor;
    }
    return std::make_pair(power, n);
}

// Recursive case.
template <std::uint64_t N>
struct PrimeFactorizationImpl {
    static_assert(N > 1);

    static constexpr std::uint64_t first_base = std::first_factor(N);
    static constexpr std::pair<std::uint64_t, std::uint64_t>
        power_and_remaining = extract_power(N, first_base);
    static constexpr auto power() { return power_and_remaining.first; }
    static constexpr auto remaining() { return power_and_remaining.second; }

    using type = typename Concatenate<
        Magnitude<BasePower<first_base, std::ratio<power()>>>,
        typename PrimeFactorizationImpl<remaining()>::type>::type;
};

// Base case.
template <>
struct PrimeFactorizationImpl<1u> {
    using type = Magnitude<>;
};

template <std::uint64_t N>
using PrimeFactorization = typename PrimeFactorizationImpl<N>::type;

```

The advantage of this format is that it's very easy to perform products and rational powers of numbers represented in this way. Indeed, at least two major C++ units libraries depend critically on this representation.

Finally, checking whether a number is prime would become extremely simple with `std::first_factor`:

```

constexpr bool is_prime(std::uint64_t n) {
    return n > 1 && std::first_factor(n) == n;
}

```

That said, it may be worth including `std::is_prime` in the standard library as well, because a dedicated primality test can be done much more quickly, in polynomial time.

§ 3.3 Open questions

The above core requirement does not fully specify this function. There are other open questions that we must consider before standardizing. However, the answers are generally less important: the integer factorization use case wouldn't depend on the details for any of them.

§ 3.3.1 Contract for numbers less than 2

The fundamental theorem of arithmetic applies only to integers greater than 1. Unsigned types have two more possible inputs, 1 and 0, and we must decide what to return for them. There are two natural choices: return the number itself, or return the multiplicative identity, 1. This paper takes no position as to which is better.

§ 3.3.2 Contract for negative numbers

It's natural to consider providing overloads for other integral types. If any of them are signed, then we have to decide how to handle negative inputs. Here are three possibilities:

1. Return -1.
2. Return the negative of the smallest prime factor of the absolute value.
3. Return the smallest prime factor of the absolute value.

This paper takes no position as to which is best.

§ 3.3.3 Larger integer types

The difficulty of integer factorization grows very rapidly with the number of bits. Although it is known to be sub-exponential, no polynomial-time algorithm has ever been discovered, and most experts believe that none exists. Additionally, the C++ language does not specify the maximum number of bits in `std::uintmax_t`. Thus, we can't promise to provide a robust and efficient implementation for “any unsigned integral type” on every platform that C++ supports. We will have to choose a maximum number of bits, and provide overloads for every integral type up to that size.

We already know that we can satisfy our goals for `std::uint64_t`. The comments in the source code for `factor.c` suggest that it also supports 128-bit integers, although we have not investigated the efficiency and robustness of this implementation.

Ultimately, we may be well served by a conservative approach, where we begin with a maximum size of 64 bits, and consider expanding the maximum in future C++ standard versions if we find that we are able to do so.

§ 4 What if we don't add this?

As we explained above, the standard units and quantities library requires this function. If we don't formally add it to the standard on its own, then we'll effectively be forced to do all the same work, but as a hidden implementation detail of the standard units library. This would make the library meaningfully harder to implement. It would also mean that other projects that require integer factorization would be unable to take advantage of the work, even though it's already done and included (although hidden) in their standard library.

Even that may not be enough. It's possible that even the ideal implementation will run into compiler iteration limits for certain inputs. If that turns out to be true, then declining to add this function means that we can't have a standard units library, either — at least, not one that supports every unit.

§ 5 Conclusion

Integer factorization is perhaps the most basic and useful operation in all of number theory. While basic implementations are easy to write, robust and efficient implementations are tricky and subtle. If results are required at compile time, the difficulty increases even more, because compilers limit the number of algorithmic iterations. In fact, there's a real risk that a `constexpr` function that produces results for every 64-bit input is not even possible for end user code!

Providing a high quality `first_factor` implementation in the C++ standard library would pave the way for a future standard units and quantities library. (Without it, the latter is at least much harder to write, and may even be impossible.) But that's not the only reason to desire it: given the wide diversity of use cases for integer factorization, we expect it would also enable many more applications that we can't yet anticipate.

§ 6 References

[Au] The Au Units library.

<https://aurora-opensource.github.io/au>

[`is_trivially_constructible` discussion] Discussion on `is_trivially_constructible` implementation (StackOverflow).

<https://stackoverflow.com/q/49901309/15777264>

[mp-units] mp-units - A Physical Quantities and Units library for C++.

<https://mpusz.github.io/mp-units>

[P3045] Quantities and units library.

<https://wg21.link/p3045>

[vector space discussion] Vector Space Representations.

https://aurora-opensource.github.io/au/0.3.4/discussion/implementation/vector_space/